QuillAudits

AUDIT REPORT

April 2026

For

fly

Table of Content

Executive Summary

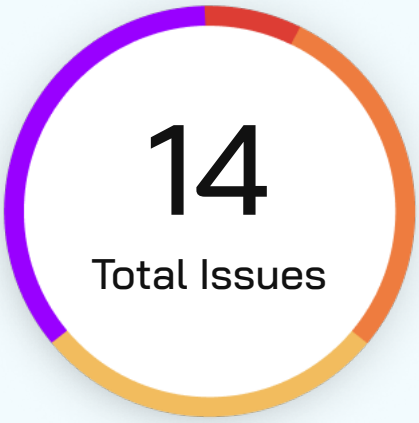
Project Name	Fly Trade
Protocol Type	Dex Aggregator
Project URL	https://www.fly.trade/
Overview	FlyTrade is a custom DEX aggregator router library that combines swap execution, dynamic multi-asset fee handling, and signature-based authorization using a highly gas-optimized but complex assembly-driven calldata decoding approach; it implements a non-standard EIP-712–style verification with internal caller whitelisting, includes basic protections like deadline checks, but introduces higher audit risk due to manual parsing, intricate memory handling, and the apparent absence of explicit replay protection, making correctness and security of offsets, signature logic, and access control critical.
Audit Scope	The scope of this Audit was to analyze the FlyTrade Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/magpieprotocol/magpie-contracts/tree/70e59c299ec253784145e485ace30fddce4a2ddd
Contracts in Scope	contracts/DexAggregator.sol contracts/DexAggregatorCore.sol contracts/libraries/LibAsset.sol contracts/libraries/LibRouter.sol
Commit Hash	70e59c299ec253784145e485ace30fddce4a2ddd
Language	Solidity
Blockchain	Ethereum, Polygon, BSC, Avalanche, Arbitrum, Optimism, Polygon zkEVM, Base, zkSync Era, Blast, Manta, Scroll, Metis, Fantom, Taiko, Sonic, Ink, Linea, Berachain, Abstract, Unichain, Monad, Stable, MegaETH, Morph, 0G, Katana, Telos, HyperEVM, Plasma
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	30th March 2026 - 9th April 2026

Updated Code Received	16th April 2026
Review 2	16th April 2026
Fixed In	https://github.com/magpieprotocol/magpie-contracts/pull/323

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.0%)
High	1 (7.15%)
Medium	4 (28.57%)
Low	4 (28.57%)
Informational	5 (35.71%)

Severity

Issues

	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	**2**	**3**	**3**
Partially Resolved	0	0	0	0	0
Resolved	0	**1**	**2**	**1**	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Unsigned maxRetentionBps and transferFromRouter fields enable complete theft of protocol retention revenue	High	Resolved
2	Unconsumed input tokens remain stuck in DexAggregator when transferFromRouter is enabled	Medium	Acknowledged
3	Missing msg.value validation traps excess native ETH in DexAggregator permanently	Medium	Resolved
4	Asymmetric fund rescue design traps unconsumed input tokens in DexAggregatorCore permanently	Medium	Acknowledged
5	Missing msg.value validation in native fee transfers enables users to avoid protocol fees	Medium	Resolved
6	Missing Uniswap V3 pool factory validation in fallback callback handler	Low	Acknowledged
7	Absence of timelocks on all admin functions enables instant privilege escalation and atomic fund extraction	Low	Acknowledged
8	Missing return data length validation in allowance enables stale memory reads as token approvals	Low	Resolved

Issue No.	Issue Title	Severity	Status
9	Short return data in isSuccessfulCall triggers EVM halt blocking non-standard token transfers	Low	Acknowledged
10	isSuccessful function Dead code in LibAsset, increases deployment gas and audit surface unnecessarily	Informational	Resolved
11	Incorrect Abs128 implementation produces mathematically wrong results for negative int128 values	Informational	Resolved
12	Unsigned sub(j, 1) underflow in executeCommandMath reads stale memory as computation result	Informational	Acknowledged
13	Identical sub(j, 1) underflow in executeCommandComparison reads stale memory as comparison result	Informational	Acknowledged
14	Misleading estimateSwapGas function name implies simulation but executes real state-changing swaps	Informational	Acknowledged

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

Unsigned maxRetentionBps and transferFromRouter fields enable complete theft of protocol retention revenue

Resolved

Path

contracts/DexAggregator.sol:151, contracts/libraries/LibRouter.sol:177

Function

`swap(), verifyBackendSignature(), LibRouter.verifySignature()`

Description

In contracts/libraries/LibRouter.sol:177-250, the verifySignature function constructs an EIP-712 message hash from SwapData fields to validate backend signatures. The function reads nine fields into the signed message: router, sender, toAddress, fromAssetAddress, toAssetAddress, deadline, amountOutMin, expectedAmountOut, and consumerId. The production type hash 0x8787ce4b243e27782d9f0fa163e1e241a55482d202acdb2625b8927c17fd7d6b corresponds to this nine-field structure. Meanwhile, LibRouter.getData parses twelve fields from calldata into the SwapData struct, including amountIn at offset 96, maxRetentionBps at offset 256, and transferFromRouter at offset 288.

However, these three additional fields are never included in the signed message hash. The maxRetentionBps field controls protocol fee retention on positive slippage in DexAggregator.sol:219-229, while transferFromRouter controls which code path executes for output token distribution in line 210-230. When transferFromRouter is false, the protocol applies retention via Math.max(expectedAmountOut, transferredAmountOut - ((transferredAmountOut * maxRetentionBps) / 10000)). When true, the contract transfers the entire transferredAmountOut directly to the user without any retention calculation. Since neither field is signed, a user can modify these bytes in the calldata after receiving a valid backend signature without invalidating that signature.

Consequently, any user can steal 100% of protocol retention revenue on every swap. An attacker receives signed calldata from the backend API with maxRetentionBps = 500 (5% fee), modifies the value to 0 in the raw calldata bytes, and submits the transaction. The signature verification passes because maxRetentionBps is not part of the signed digest. For a swap where the DEX returns 110 tokens with expectedAmountOut = 100, the legitimate flow would give the user 104.5 tokens and the protocol 5.5 tokens, but the attack flow gives the user all 110 tokens and the protocol zero. Alternatively, the attacker can flip transferFromRouter from false to true, causing line 212 to execute transfer(toAddress, transferredAmountOut) directly, bypassing the retention logic entirely. At $10M daily volume with 5% retention, this vulnerability causes $500,000 daily loss or $182.5M annual loss. A sophisticated attacker combines both manipulations for redundant bypass, ensuring zero protocol revenue regardless of code path.

Proof of Concept

The attack proceeds as follows.

1. The user requests a swap quote from the Magpie backend, which returns signed calldata containing maxRetentionBps = 500 and transferFromRouter = false.
2. Before submitting the transaction, the user locates the maxRetentionBps bytes at calldata offset 205-208 and overwrites them with zeros.
3. The user submits the modified calldata to DexAggregator.swapWithBackendSignature.
4. LibRouter.verifySignature computes the EIP-712 digest using only the nine signed fields and recovers the backend signer address, which matches the authorized internal caller.
5. The swap executes with maxRetentionBps = 0, causing the retention calculation to yield zero: 110 - ((110 * 0) / 10000) = 110.
6. The user receives 110 tokens instead of 104.5 tokens, stealing 5.5 tokens of protocol revenue.

For monetary verification with concrete values: DEX output is 110 tokens, expected output is 100 tokens, and intended retention is 5%. Legitimate calculation yields max(100, 110 - 5.5) = 104.5 tokens to user and 5.5 tokens to protocol. Attack calculation yields max(100, 110 - 0) = 110 tokens to user and 0 tokens to protocol. The stolen amount per swap is 5.5 tokens, representing 100% of protocol revenue.

Recommendation

We recommend including amountIn, maxRetentionBps, and transferFromRouter in the EIP-712 type hash by updating the type string to

```
Swap(address router,address sender,address recipient,address fromAsset,address toAsset,uint256 deadline,uint256 amountOutMin,uint256 expectedAmountOut,bytes32 consumerId,uint256 amountIn,uint256 maxRetentionBps,bool transferFromRouter)
```

and adding the corresponding mstore instructions in verifySignature at offsets 320, 352, and 384 to include these fields in the message hash computation, then updating messageLength from 320 to 416 bytes.

Medium Severity Issues

Unconsumed input tokens remain stuck in DexAggregator when transferFromRouter is enabled

Acknowledged

Path

contracts/DexAggregator.sol:168-169, 211-218

Function Name

`Swap`

Description

In contracts/DexAggregator.sol:168-169, the swap function handles the transferFromRouter=true code path by pulling the full amountIn from the user into the DexAggregator contract and then approving the Core contract for that same amount. This design allows the Core to consume tokens on behalf of the user through the aggregator's approval rather than directly from the user.

However, after Core execution completes, lines 211-218 calculate the unconsumed portion by checking the remaining allowance between the aggregator and Core. The code correctly tracks this value in the approval variable and uses it to compute transferredAmountIn for event emission. When approval > 0, the code resets the Core's allowance to zero to prevent future unauthorized spending. The critical flaw is that no corresponding transfer call exists to return the unconsumed tokens to the user. The approval value is computed, the allowance is cleared, but the actual token balance representing those unconsumed tokens remains permanently in the DexAggregator contract.

Consequently, users lose (amountIn - consumed) tokens on every swap where the Core does not fully utilize the approved amount. This scenario occurs legitimately when limit orders are partially filled, when AMM price movements cause less favorable execution, or when aggregation routes consume fewer tokens than initially estimated. The only recovery mechanism requires a privileged fundManager to manually call transfer on the aggregator, which provides no guarantee that users will be made whole. At scale, assuming 1% of daily volume experiences partial consumption with an average 2% unconsumed rate on $10M daily volume, users collectively lose approximately $2,000 per day or $730,000 annually.

Proof of Concept

Attack steps are as follows:

1. User approves and initiates swap with amountIn = 100 tokens
2. DexAggregator pulls 100 tokens from user (line 168)
3. DexAggregator approves Core for 100 tokens (line 169)
4. Core executes and only consumes 98 tokens
5. Aggregator calculates approval = 2 (unconsumed)
6. Aggregator resets Core approval to zero (line 217)
7. Result: 2 tokens stuck in Aggregator, user balance shows loss of 100 (not 98)

```
// Line 211-218 - approval tracked but not refunded
uint256 approval = swapData.fromAssetAddress.allowance(address(this),
currentCoreAddress);
transferredAmountIn = swapData.amountIn - approval;

if (approval > 0) {
    swapData.fromAssetAddress.approve(currentCoreAddress, 0);  // ✓ Done
}
// MISSING: swapData.fromAssetAddress.transfer(fromAddress, approval);  // ✗
Not done
```

Recommendation

We recommend adding a token transfer to refund unconsumed tokens immediately after resetting the Core approval by modifying lines 215-218 to include swapData.fromAssetAddress.transfer(fromAddress, approval) within the approval > 0 conditional block, ensuring users receive their unconsumed tokens in the same transaction as the swap execution.

```
if (approval > 0) {
    swapData.fromAssetAddress.approve(currentCoreAddress, 0);
    swapData.fromAssetAddress.transfer(fromAddress, approval);  // Refund
unconsumed
}
```

FlyTrade Team's Comment

This is built intentionally like this to collect positive slippage in the fromAsset

Missing msg.value validation traps excess native ETH in DexAggregator permanently

Resolved

Path

contracts/DexAggregator.sol:264

Function Name

`adjustAmountIn()`

Description

In contracts/DexAggregator.sol:264-276, the adjustAmountIn function handles automatic derivation of the swap amount when swapData.amountIn equals zero. For native ETH swaps in auto-mode, the function correctly computes amountIn as msg.value - transferredFeeAmount, ensuring exact consumption of sent funds. For ERC20 swaps in auto-mode, it derives the amount from the minimum of the user's allowance and balance.

However, when a user provides an explicit non-zero amountIn value, the function bypasses all validation logic entirely. There is no check ensuring that msg.value >= amountIn + transferredFeeAmount for native ETH swaps. Subsequently, in adjustFromAssetAddress at lines 257-261, only the exact amountIn is wrapped to WETH via wrap(swapData.amountIn), leaving any excess ETH in the DexAggregator contract. The contract's receive function accepts arbitrary ETH deposits but provides no refund mechanism for overpayments.

Consequently, users who send msg.value exceeding amountIn + fees permanently lose the excess ETH to the DexAggregator contract. For example, if a user sends 10 ETH with amountIn = 5 ETH and fees of 1 ETH, the 4 ETH excess remains trapped with no automated recovery path. While under-payment (msg.value < amountIn) safely reverts during the wrap call due to insufficient balance, over-payment silently succeeds and the user receives no error or refund. The only recovery mechanism requires a privileged fundManager to manually execute transfer, which provides no guarantee of user reimbursement and no way to identify which user's funds are trapped. At current ETH prices, a single 4 ETH overpayment represents approximately $12,000 in permanent user loss.

Proof of Concept

The attack proceeds as follows.

1. User initiates swap with msg.value = 10 ETH, amountIn = 5 ETH, fees = 1 ETH
2. LibRouter.transferFees sends 1 ETH to fee receiver (line 110)
3. adjustAmountIn skips validation because amountIn != 0 (line 265)
4. adjustFromAssetAddress wraps exactly 5 ETH to WETH (line 260)
5. Result: 4 ETH excess (10 - 1 - 5) permanently stuck in DexAggregator

```
function adjustAmountIn(SwapData memory swapData, uint256 transferredFeeAmount)
private {
    if (swapData.amountIn == 0) {  // Only auto-mode triggers
        if (swapData.fromAssetAddress.isNative()) {

            swapData.amountIn = msg.value - transferredFeeAmount;  // Safe:
exact
        }
    }
    // MISSING: else { validate msg.value and refund excess }
}
```

Recommendation

We recommend adding explicit validation and refund logic for native ETH swaps when amountIn is non-zero by inserting a check after the existing auto-mode block that verifies msg.value >= swapData.amountIn + transferredFeeAmount and transfers any excess back to the sender, as shown below:

```
function adjustAmountIn(SwapData memory swapData, uint256 transferredFeeAmount)
private {
    if (swapData.amountIn == 0) {

        if (swapData.fromAssetAddress.isNative()) {

            swapData.amountIn = msg.value - transferredFeeAmount;

        } else {

         uint256 allowance = swapData.fromAssetAddress.allowance(msg.sender,
address(this));

       uint256 balance = swapData.fromAssetAddress.getBalanceOf(msg.sender);

            swapData.amountIn = allowance < balance ? allowance : balance;
        }

    } else if (swapData.fromAssetAddress.isNative()) {

        require(msg.value >= swapData.amountIn + transferredFeeAmount,
"InsufficientValue");

        uint256 excess = msg.value - swapData.amountIn - transferredFeeAmount;

        if (excess > 0) {
            payable(msg.sender).transfer(excess);

        }
    }
}
```

Asymmetric fund rescue design traps unconsumed input tokens in DexAggregatorCore permanently

Acknowledged

Path

contracts/DexAggregator.sol:151

Function Name

`swap()`

Description

In contracts/DexAggregator.sol:171-173, when transferFromRouter=false, the swap function transfers input tokens directly from the user to DexAggregatorCore via swapData.fromAssetAddress.transferFrom(fromAddress, coreAddress, swapData.amountIn). This bypasses the DexAggregator contract entirely, sending the full amountIn straight to the Core contract.

However, if the external DEX pool does not fully consume the input tokens due to liquidity constraints, multi-hop rounding, or price movement between signing and execution, the unconsumed remainder becomes trapped in DexAggregatorCore. Unlike the transferFromRouter=true path in lines 167-170, which routes tokens through DexAggregator and tracks unconsumed amounts via allowance differentials in lines 214-218, the transferFromRouter=false path in lines 220-230 performs no tracking of unconsumed input tokens whatsoever. The code comment on line 220 states “Slippage in toAsset”, confirming this path only handles output-side slippage and ignores input-side under-consumption.

Consequently, the design creates an asymmetric rescue mechanism gap between the two contracts. DexAggregator provides a simple transfer function gated by fundManager that enables straightforward fund recovery. DexAggregatorCore, by contrast, offers no equivalent transfer function, verified by examining IDexAggregatorCore.sol which defines only setWhitelist and multicall. The sole rescue path requires the protocol owner to construct complex delegatecall sequences via multicall, a mechanism designed for batch operations rather than fund recovery. For a 1,000 USDC swap where the external DEX consumes only 950 USDC, the remaining 50 USDC (~$50) becomes trapped until protocol intervention.

Proof of Concept

1. User initiates a swap with transferFromRouter=false for 1,000 USDC as input
2. DexAggregator calls transferFrom(user, coreAddress, 1000 USDC) in line 172
3. Core executes the swap via external DEX, which consumes only 950 USDC due to pool constraints
4. Post-swap logic in lines 220-230 handles output distribution but never checks input consumption
5. 50 USDC remains in DexAggregatorCore with no user-accessible recovery
6. Owner must construct a delegatecall payload for multicall to rescue, requiring technical expertise

Recommendation

We recommend adding a transfer function to DexAggregatorCore gated by onlyOwner to provide symmetric rescue capability with DexAggregator, implemented as

```
function transfer(TransferParam[] calldata transfers) external onlyOwner {
for (uint256 I; I < transfers.length; I++)
{          transfers[I].assetAddress.transfer(transfers[I].toAddress,
transfers[I].amount);
 }
}
```

using the existing TransferParam struct pattern from IDexAggregator.sol.

FlyTrade Team's Comment

This is built intentionally like this, it's on us to use all of the amount which is sent to the Core contract and leave nothing unconsumed. We don't think this is an issue.

Missing msg.value validation in native fee transfers enables users to avoid protocol fees

Resolved

Path

contracts/libraries/LibRouter.sol:108-110

Function Name

`transferFees()`

Description

In contracts/libraries/LibRouter.sol:108-110, the transferFees function handles native ETH fee transfers by calling LibAsset.transfer, which executes a low-level call sending ETH from the contract balance. In contracts/libraries/LibAsset.sol:117-122, the transfer function for native assets performs payable(recipient).call{value: amount}(“”), drawing from address(this).balance rather than from msg.value.

However, there is no validation that msg.value covers the total native fee amount when the swap input asset is an ERC20 token. The DexAggregator contract can accumulate ETH through multiple vectors: the receive() external payable {} function accepts any direct ETH transfers, slippage dust remains from swap execution, users may accidentally send ETH directly to the contract, and cross-chain bridge refunds may deposit ETH. When ETH accumulates in the contract, users can submit swaps with msg.value = 0 while the backend-signed fee parameters specify native ETH fees, causing the protocol to pay fees from its own accumulated balance instead of requiring payment from the user.

Consequently, users can systematically avoid paying protocol fees by timing their swaps when the contract has accumulated ETH balance. For a swap with a 1 ETH native fee requirement, a user submitting msg.value = 0 receives a “free” swap where the fee is paid from contract-held ETH. While the fee receiver remains the legitimate protocol wallet (as signed by the backend), the protocol loses revenue equal to the accumulated balance that users exploit.

Recommendation

We recommend adding explicit msg.value validation at the start of swapWithBackendSignature by calculating total native fees from swapData.swapFeeAmounts where swapData.swapFeeAssetAddresses[I] == address(0), adding the native swap input amount if fromAssetAddress is native, and reverting with InsufficientNativeValue if msg.value is less than the computed required native amount.

Low Severity Issues

Missing Uniswap V3 pool factory validation in fallback callback handler

Acknowledged

Path

contracts/DexAggregatorCore.sol:81

Function Name

`fallback()`

Description

In contracts/DexAggregatorCore.sol:81-107, the fallback function handles Uniswap V3 swap callbacks by checking that calldatasize() == 164 and then extracting amount0Delta, amount1Delta, and assetIn from fixed calldata offsets before executing a token transfer to msg.sender. The function relies solely on the 164-byte size check to identify legitimate Uniswap V3 callback requests.

However, the function does not validate that msg.sender is a legitimate Uniswap V3 pool deployed by the canonical factory. Any external address can craft a 164-byte calldata payload that passes the size check, sets a positive amount0Delta or amount1Delta, and specifies an arbitrary token address at offset 132. The fallback then executes assetIn.transfer(msg.sender, amount) without verifying the caller's authenticity. This deviates from the standard Uniswap V3 integration pattern where callbacks verify the pool address using CREATE2 computation from the factory, token pair, and fee tier.

Consequently, if the DexAggregatorCore contract holds any token balance, an attacker can drain those tokens by calling the fallback with a crafted payload. The protocol's design assumption, explicitly stated in the contract comment "this contract is not supposed to store tokens," means this vulnerability has limited practical impact under normal operation. The Core contract does not accumulate tokens during atomic swap execution because input tokens are swapped through DEXs and output tokens are transferred to the DexAggregator. Token accumulation would require abnormal scenarios such as direct user transfers by mistake, fee-on-transfer token dust, or misconfigured backend command sequences that leave residual balances. Given these prerequisites, the issue represents a defense-in-depth concern rather than an immediately exploitable vulnerability.

Recommendation

We recommend adding factory validation by storing the Uniswap V3 factory address and computing the expected pool address using CREATE2 with the init code hash, then comparing against msg.sender before executing any transfer, or alternatively implementing an execution lock by setting a storage flag at the start of executeCommandCall and clearing it afterward while checking this flag in fallback to ensure callbacks only execute during active command sequences.

FlyTrade Team's Comment

Acknowledged: it doesn't matter for us. You mentioned already that it's not applicable in normal operations. Fixing this would just cost additional gas to the users without adding any benefits.

Absence of timelocks on all admin functions enables instant privilege escalation and atomic fund extraction

Acknowledged

Description

In contracts/DexAggregator.sol:45-57 and contracts/DexAggregatorCore.sol:55, all administrative functions execute immediately without timelocks, multi-signature requirements, or delay mechanisms. The affected functions include updateCoreAddress at line 45, updateInternalCaller at line 50, updateFundsManager at line 57, and updateWhitelist on DexAggregatorCore at line 55. Each function modifies critical protocol state in a single transaction, and the multicall function at line 89 enables batching multiple admin operations atomically via delegatecall, preserving msg.sender as the owner throughout the sequence.

However, this architecture creates multiple attack vectors that share the same root cause. First, the backend signer controls all swap parameters including amountOutMin and routing data, and updateInternalCaller allows instant signer rotation with no delay, enabling a compromised or malicious signer to sign swaps with amountOutMin = 1 wei for near-total sandwich extraction before being removed in the same block. Second, updateCoreAddress executes immediately, allowing the owner to front-run pending user swaps by replacing the core address with a malicious contract that captures tokens, then restoring the legitimate core afterward. Third, updateWhitelist on DexAggregatorCore grants immediate command execution privileges to any address, enabling whitelisted attackers to execute arbitrary external calls, set persistent ERC20 approvals, or directly drain funds through the command engine. Fourth, the multicall function combined with updateFundsManager enables atomic privilege escalation sequences where the owner grants fund manager role, drains all tokens via transfer, and revokes the role in a single transaction.

Consequently, users have no on-chain protection against administrative actions occurring between their transaction submission and execution. A compromised owner key enables complete theft of all swap volume during the attack window. The atomic nature of these attacks via multicall means monitoring systems cannot detect intermediate state changes, and the cleanup steps leave minimal forensic evidence beyond event logs. While event emissions do record all actions permanently on-chain, real-time detection is impractical given block-level atomicity.

Recommendation

We recommend implementing a unified timelock mechanism across all administrative functions by adding a pendingChange mapping and changeQueuedAt timestamp for each function, requiring a minimum 1-hour delay between queueing and execution for updateCoreAddress, updateInternalCaller, updateWhitelist, and updateFundsManager, emitting events at queue time to enable monitoring systems to detect and alert on pending changes, and modifying multicall to prevent batching role-change selectors with fund-transfer selectors in the same call to eliminate atomic privilege escalation vectors.

FlyTrade Team's Comment

We acknowledge this finding. We do not believe timelocks alone solve the stated risk. Actual mitigation is is to make a multisig owner (N-of-M), which eliminates the single-key failure mode.

Missing return data length validation in allowance enables stale memory reads as token approvals

Resolved

Path

contracts/libraries/LibAsset.sol:170

Function Name

`allowance()`

Description

In contracts/libraries/LibAsset.sol:170-186, the allowance function performs a staticcall to retrieve an ERC20 token's approval amount. The function allocates a 32-byte output buffer and executes the call, then immediately reads the result from memory via mload(currentOutputPtr) without verifying that the call actually returned data.

However, the function does not validate returndatasize() before reading the output buffer. When the target address is not an ERC20 contract (an EOA or a contract without an allowance function), the staticcall may succeed with zero return bytes. In this scenario, the EVM writes nothing to the output buffer, and the subsequent mload reads whatever stale data previously occupied that memory location. This stale value is then interpreted as the token's allowance amount.

Consequently, if the backend provides an incorrect fromAssetAddress during auto-mode swaps (where amountIn == 0), the protocol may read an arbitrary memory value as the user's allowance. This could cause the subsequent transferFrom to attempt moving an incorrect token amount, leading to unexpected transaction failures or, in edge cases where stale memory contains a value matching actual user balances, unintended token transfers. The vulnerability requires backend misconfiguration and cannot be directly exploited by users, limiting its practical impact.

Recommendation

We recommend adding a returndatasize() validation check after the staticcall in LibAsset.allowance by inserting if lt(returndatasize(), 32) { amount := 0 } before the mload operation, ensuring that calls returning fewer than 32 bytes are treated as having zero allowance rather than reading potentially stale memory values.

Short return data in isSuccessfulCall triggers EVM halt blocking non-standard token transfers

Acknowledged

Path

contracts/libraries/LibAsset.sol:208

Function Name

`execute(), isSuccessfulCall()`

Description

In contracts/libraries/LibAsset.sol:208-231, the execute function calls the internal isSuccessfulCall helper to determine whether an ERC20 transfer or approve operation succeeded. When returndatasize() is greater than zero, the function executes returndatacopy(0, 0, 32) to copy 32 bytes of return data into memory for inspection.

However, the function assumes that any non-zero return data will be at least 32 bytes long. If a non-standard token returns between 1 and 31 bytes, the returndatacopy(0, 0, 32) instruction attempts to copy more data than exists in the return buffer. This violates the EVM's bounds checking rules, causing an immediate exceptional halt that reverts the entire transaction. The code lacks a conditional check for returndatasize() < 32 before attempting the 32-byte copy operation.

Consequently, any ERC20 token that returns a non-standard response length between 1 and 31 bytes becomes completely incompatible with the DexAggregator protocol. All transfer and approve operations involving such tokens will unconditionally revert, preventing users from swapping these tokens entirely. While standard ERC20 implementations return either 0 bytes (USDT-style) or 32 bytes (standard bool), edge-case tokens with atypical return lengths would be silently blocked with no clear error message. This represents a compatibility limitation rather than a direct security vulnerability, as no funds are at risk and standard tokens function correctly.

Recommendation

We recommend modifying the isSuccessfulCall function to explicitly handle return data lengths using a switch statement that checks for exactly 0 bytes (USDT-style success), exactly 32 bytes (standard boolean return), and treats all other lengths as failures by returning isSuccessful := 0, thereby preventing the out-of-bounds returndatacopy operation while maintaining compatibility with both standard and non-compliant ERC20 tokens.

FlyTrade Team's Comment

As a theoretical EVM edge case. No deployed token with meaningful liquidity returns between 1 and 31 bytes from ERC-20 calls, we find this to be non-issue.

Informational Issues

isSuccessful function Dead code in LibAsset, increases deployment gas and audit surface unnecessarily

Resolved

Path

contracts/libraries/LibAsset.sol:192

Function Name

`isSuccessful()`

Description

In contracts/libraries/LibAsset.sol:196-208, the isSuccessful The “isSuccessful” function in LibAsset.sol is redundant dead code that is never invoked by any other contract. Its presence unnecessarily increases deployment gas costs and expands the codebase’s audit surface area

Recommendation

We recommend removing the dead isSuccessful function from LibAsset.sol to reduce deployment gas costs and eliminate unnecessary audit surface, retaining only the inline isSuccessfulCall Yul function within execute which provides the same functionality with better gas efficiency.

Incorrect Abs128 implementation produces mathematically wrong results for negative int128 values

Resolved

Path

contracts/DexAggregatorCore.sol:506

Function Name

`executeCommandMath() — operator case 6`

Description

In executeCommandMath (lines 578–584), the Abs128 operation incorrectly computes the absolute value by applying a 256-bit arithmetic shift (sar) on a value intended to be treated as a 128-bit signed integer. Since the EVM checks the 256-bit sign bit (bit 255) instead of the 128-bit sign bit (bit 127), negative int128 values (e.g., -1 stored as $2^{128}-1$) are misinterpreted, producing an incorrect mask of 1 instead of all-ones. This leads to faulty XOR and SUB operations, resulting in massively incorrect outputs (e.g., abs(-1) becomes $2^{128}-3$ instead of 1). The root cause is improper handling of signedness and bit width mismatch between int128 semantics and uint256 operations.

As a result, any downstream logic relying on this value—such as transfers or calls—can execute with corrupted amounts, potentially causing severe financial inconsistencies or unexpected reverts.

Recommendation

We recommend using signextend(15, amount0) before applying the mask to properly sign-extend the int128 value to int256, then performing sar(255, amount0) on the sign-extended value to produce the correct all-ones mask for negative inputs, as shown: amount0 := signextend(15, amount0) followed by let mask := sar(255, amount0) with the existing XOR and SUB operations.

Unsigned sub(j, 1) underflow in executeCommandMath reads stale memory as computation result

Acknowledged

Path

contracts/DexAggregatorCore.sol:558

Function Name

`executeCommandMath() -> math() assembly function, case 0 terminator`

Description

In contracts/DexAggregatorCore.sol:558-563, the math Yul function within executeCommandMath uses a None operator (case 0) as a loop terminator to signal the end of a mathematical operation sequence. When this terminator is encountered, the function computes finalPtr by adding currentOutputPtr to mul(sub(j, 1), 32), where j is the current iteration index. The function then reads the result from finalPtr and updates the free memory pointer.

However, when the None operator appears as the first instruction at j = 0, the expression sub(j, 1) underflows in EVM unsigned arithmetic. The EVM computes sub(0, 1) as type(uint256).max (2^256 - 1). Subsequently, mul(type(uint256).max, 32) produces 2^256 - 32, which in two's complement representation equals -32. Adding this to currentOutputPtr wraps the pointer backward by 32 bytes, causing finalPtr to point to memory 32 bytes before the intended output buffer location.

mload(finalPtr) reads whatever stale data occupied that memory location from previous operations. In the execute context, this stale value becomes the "math result" stored at outputOffsetPtr. If subsequent commands such as Transfer or Call consume this garbage value as an amount parameter, the protocol could execute token transfers with arbitrary amounts derived from uninitialized or stale memory. Since commands are not signed (only swap metadata is), a whitelisted caller or compromised backend can craft calldata sequences that trigger this underflow, potentially propagating garbage values into financial operations.

Recommendation

We recommend adding an iszero(j) guard at the beginning of the case 0 handler to return a safe default value (zero) when the None operator appears at position zero, implemented as if iszero(j) { amount := 0; leave } before the existing finalPtr calculation.

FlyTrade Team's Comment

The command bytes are not covered by the EIP-712 signature, so this is user-triggerable. However the impact is limited to the caller wasting their own gas

Identical sub(j, 1) underflow in executeCommandComparison reads stale memory as comparison result

Acknowledged

Path

contracts/DexAggregatorCore.sol:671

Function Name

`executeCommandComparison() -> comparison() assembly function, case 0 terminator`

Description

In contracts/DexAggregatorCore.sol:690-695, the comparison Yul function within executeCommandComparison contains the identical underflow pattern as the math function described in N-01. The None operator (case 0) serves as a loop terminator, computing finalPtr via add(currentOutputPtr, mul(sub(j, 1), 32)) and reading the result with mload(finalPtr).

However, when the None operator appears at iteration j = 0, the same unsigned arithmetic underflow occurs. The expression sub(0, 1) produces type(uint256).max, and the subsequent multiplication and addition cause finalPtr to wrap backward by 32 bytes. The mload operation then reads memory from before the intended output buffer, returning stale data as the comparison result.

Consequently, the comparison command propagates garbage values to subsequent operations in the same manner as the math command underflow. This represents a duplicate vulnerability sharing the same root cause, exploitation prerequisites, and impact profile as above issue. A whitelisted caller can craft command sequences where the Comparison command with a None terminator at position zero produces arbitrary values that flow into Transfer or Call commands, potentially causing incorrect token movements based on stale memory contents.

Recommendation

We recommend applying the same iszero(j) guard as recommended for above issue by adding if iszero(j) { amount := 0; leave } at the beginning of the case 0 handler in the comparison function to prevent the underflow and return a safe default value.

Misleading estimateSwapGas function name implies simulation but executes real state-changing swaps

Acknowledged

Path

contracts/DexAggregator.sol:278-287

Function Name

`estimateSwapGas()`

Description

In contracts/DexAggregator.sol:278-287, the estimateSwapGas function is defined as an external payable function that returns both amountOut and gasUsed. The function name suggests gas estimation or simulation behavior, which in standard Ethereum development implies a read-only operation that does not modify state. The function executes the identical code path as swapWithBackendSignature at lines 290-298, calling verifyBackendSignature, LibRouter.transferFees, adjustAmountIn, adjustFromAssetAddress, and swap in sequence.

However, the function performs real state changes including token transfers, fee deductions, and swap execution rather than simulating these operations. Unlike DexAggregatorCore.simulate which intentionally reverts after execution to prevent state persistence, estimateSwapGas commits all changes. The only difference from swapWithBackendSignature is that estimateSwapGas returns the gasUsed value from Core's internal tracking while swapWithBackendSignature discards it. Both functions require a valid backend signature, which means accidental execution without intentional action is not possible since users must explicitly request and receive a signature from the backend API.

Consequently, the misleading function name creates a documentation and naming concern rather than a direct security vulnerability. Professional integrators following standard Ethereum practices use eth_call for simulation regardless of function name, which does not persist state changes. The backend signature requirement prevents "accidental" real swaps since signatures are single-use and must be explicitly obtained. The impact is limited to potential confusion for developers unfamiliar with the codebase who might incorrectly assume the function is safe to call as a transaction for testing purposes, though such usage would require them to have already obtained a valid signature.

Recommendation

We recommend renaming estimateSwapGas to swapWithGasTracking to accurately reflect that the function executes a real swap while additionally returning gas usage metrics, or alternatively removing the function entirely and adding gasUsed to the return value of swapWithBackendSignature since both functions perform identical operations.

FlyTrade Team's Comment

We use it for swap gas estimation. We don't think we need to change the name.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Fly Trade. We performed our audit according to the procedure described above.

Multiple High, Medium, Low and Informational Issues were found during the Audit. In the End, The Fly Trade Team Resolved few issues and Acknowledged the others.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

April 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com